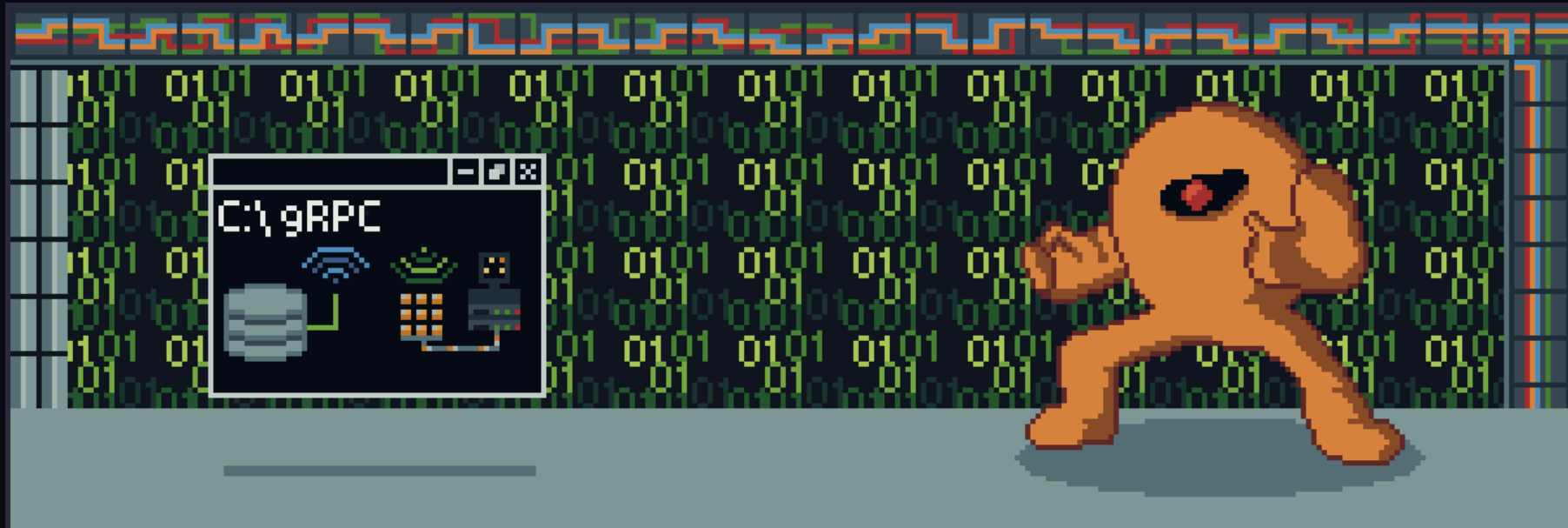




YELLOW DEVIL

O mesmo boss, **dois paradigmas** de comunicação distribuída

Ramon Couto Santos & Aaron Goldberg Guerra · Desenvolvimento de Sistemas Distribuídos

Por que um chefe de Mega Man?

O **Yellow Devil** (Mega Man, 1987) é famoso por uma coisa só: ele **se desmonta em partículas**, atravessa a tela e **se remonta do outro lado**. Só dá para acertá-lo nesse trânsito.

Isso **já é** um sistema distribuído: um estado que sai de um lugar, viaja em pedaços independentes e é reconstruído em outro.

A REGRA DO PROJETO

O sprite tem **52 × 36** e **1.328 pixels** não-vazios.

1 pixel = 1 mensagem. 1.328 mensagens = 1 boss.

Se as mensagens chegam, o monstro aparece. Se alguma se perde, fica um buraco nele. **O desenho é o log.**



O boss montado num terminal — 1.328 partículas no lugar

Mesmo boss, mesmo sprite, dois transportes

Os dois trabalhos resolvem **o mesmo problema** com tecnologias diferentes. Trocamos só **o meio pelo qual a partícula viaja** — e é essa a comparação.

	TRABALHO 1 · gRPC	TRABALHO 2 · MOM
A partícula vira	uma mensagem de um stream RPC	uma mensagem numa fila disputada
Tecnologia	Protocol Buffers + HTTP/2	RabbitMQ (AMQP + STOMP)
Quem conhece quem	cliente sabe o endereço do servidor	ninguém sabe de ninguém — só o nome da fila
Se o outro lado cai	o stream quebra — a chamada falha	a mensagem volta pra fila e é reentregue a outro
O que vamos provar	o boss se remonta no outro processo , partícula a partícula, pelo stream	666 + 662 = 1.328 — cada mensagem foi para um só consumidor
Comando	<code>devil grpc</code>	<code>devil mom</code>

Roteiro de hoje: primeiro o gRPC (chamada remota tipada), depois o MOM (fila desacoplada). No fim, comparamos os dois no mesmo problema.

PARTE 1

gRPC

Chamar um método que está **na outra máquina** como se fosse local — com contrato tipado e streaming

O que é RPC, e o que o gRPC acrescenta

RPC — REMOTE PROCEDURE CALL

Eu chamo `client.Teleport(...)`.

Parece um método local.

Executa em outro processo, em outra máquina.

Serialização, rede e protocolo ficam escondidos. É esse "esconder" que dá nome à sigla.

O QUE O gRPC ACRESCENTA

► **Contrato .proto**

um arquivo define as mensagens e as operações — os dois lados nascem dele

► **Código gerado**

o compilador escreve o stub; ninguém escreve rede à mão

► **HTTP/2 + streaming**

uma conexão só, aberta, nos dois sentidos

► **Binário (Protobuf)**

poucos bytes por mensagem, em vez de JSON

Guarde esta palavra: contrato. É dele que sai tudo — cliente, servidor e a garantia de que os dois falam a mesma língua.

Tudo começa por este arquivo

```
// A mensagem COMPOSTA: 4 atributos. A posição viaja JUNTO com o pixel.
message DevilPart {
    int32 part_id    = 1;    // a ordem do pedaço
    bytes pixel_data = 2;    // a cor daquele pixel
    int32 target_x   = 3;    // onde ele se encaixa...
    int32 target_y   = 4;    // ...do outro lado
}

// O serviço: UM serviço, DUAS operações remotas.
service YellowDevilTransfer {
    // IDA: as partes fluem do cliente para o servidor
    rpc Teleport      (stream DevilPart) returns (ReassemblyStatus);

    // VOLTAR: o cliente pede e as partes fluem de volta
    rpc TeleportBack (ReturnRequest)   returns (stream DevilPart);
}
```

Repare no stream: ele aparece na **entrada** da primeira operação e na **saída** da segunda. Esse detalhe é o que faz o boss se desmontar e se remontar animado.

Por que definimos o contrato assim?

► Por que **streaming** e não uma chamada normal?

São 1.328 partículas. Uma RPC por partícula = 1.328 idas e voltas (lento). Um `repeated` com tudo dentro = o boss aparece de uma vez, **sem animação**. O stream resolve os dois: **uma conexão**, e o servidor desenha **cada partícula à medida que chega**.

► Por que **DevilPart** carrega a posição?

Porque assim a mensagem é **auto-suficiente**: o receptor não guarda estado nenhum para saber onde encaixar o pedaço. Chegou, desenhou.

► Por que **pixel_data** é bytes e não string?

Para não amarrar o contrato a um formato de cor. Hoje é uma letra da paleta; poderia virar RGB **sem quebrar o .proto**.

► Por que devolver **ReassemblyStatus** em vez de nada?

Confirmação **tipada** no fim da ida: `is_complete + message`. O cliente sabe que o monstro chegou inteiro.

Como a comunicação acontece

CLIENTE

console .NET
Grpc.Net.Client
stub

1. `Teleport(stream DevilPart)` – 1.328 mensagens no MESMO stream

2. `ReassemblyStatus` – "reconstruído com 1328 partes!"

SERVIDOR

ASP.NET Core
Grpc.AspNetCore
porta 5254 · HTTP/2



À esquerda o **cliente** se desintegrando (“Teleportando...”) · à direita o **servidor** remontando (“Aguardando partículas...”) — **dois processos, um stream**

Detalhe que vale citar: o servidor guarda quem é o "dono" do monstro com um Lock. Se dois clientes pedirem ao mesmo tempo, **só o primeiro leva** — o outro recebe um stream vazio.

O que o compilador Protobuf gera sozinho?

Ninguém escreve nada disso à mão. O pacote **Grpc.Tools** roda o protoc **durante o dotnet build** e gera, a partir do `.proto`:

Arquivo gerado	O que tem dentro
YellowDevilServer/obj/.../Protos/YellowDevil.cs	as classes das mensagens (DevilPart, ReassemblyStatus...) + serialização
YellowDevilServer/obj/.../Protos/YellowDevilGrpc.cs	YellowDevilTransferBase — a classe que o servidor herda
YellowDevilClient/obj/.../YellowDevil.cs	as mesmas classes de mensagem, do lado do cliente
YellowDevilClient/obj/.../YellowDevilGrpc.cs	YellowDevilTransferClient — o stub que o cliente chama

Quem decide o que gerar é o GrpcServices no `.csproj`: Server num, Client no outro — **a partir do mesmo .proto**. Fonte única = os dois lados nunca discordam.

Por que gRPC e não uma API REST?

AQUI O gRPC GANHA

▶ Streaming de verdade

fluxo contínuo de 1.328 partículas numa conexão só

▶ Contrato tipado

errou o campo, **não compila** — em REST só quebra em runtime

▶ Binário compacto

DevilPart em Protobuf = poucos bytes; em JSON, várias vezes mais —
×1.328

▶ Stub pronto

sem URL, verbo, status code ou parse manual

ONDE O REST GANHARIA

▶ Consumidor genérico

se fosse qualquer navegador chamando, sem stub

▶ Chamadas esporádicas e avulsas

um CRUD comum, sem fluxo contínuo

▶ Depurar na mão

com REST dá para usar curl e ler o JSON a olho nu

Não é o nosso caso: aqui é **fluxo contínuo entre dois processos que nós controlamos**.

Como executar e conectar — gRPC

```
# Um comando abre as 3 abas:
$ ./devil.sh grpc           # Linux
> .\devil.ps1 grpc         # Windows

# O que ele faz por baixo:
$ dotnet build              # restaura + GERA OS STUBS + compila
$ dotnet run --project YellowDevilServer
$ dotnet run --project YellowDevilClient -- \
    http://localhost:5254 boss # nasce COM o monstro
$ dotnet run --project YellowDevilClient -- \
    http://localhost:5254      # nasce vazio
```

Controles: ENTER com o monstro = **teleporta pro servidor** · ENTER sem o monstro = **invoca de volta**

COMO CONECTAR

O cliente recebe **o endereço do servidor** como argumento — é só isso que ele precisa saber.

```
# outro PC, mesma rede:
dotnet run --project YellowDevilClient -- \
    http://192.168.0.10:5254 boss
```

PORTAS

5254 — clientes gRPC (HTTP/2)

5255 — modo sala: qualquer um abre

http://IP:5255 no **navegador**, sem instalar nada

No Windows, liberar antes: netsh advfirewall firewall add rule ... localport=5254,5255

PARTE 2

MOM

Middleware Orientado a Mensagens: ninguém chama ninguém — todo mundo fala
com **uma fila no meio**

O que muda quando entra um broker

NO gRPC (parte 1)

O cliente **precisa saber** onde o servidor está:

`http://192.168.0.10:5254`.

Os dois têm que estar **online ao mesmo tempo**. Se o servidor cai, a chamada **falha**.

NO MOM (parte 2)

Produtor e consumidor só conhecem **o nome da fila**.

Não sabem o endereço um do outro. **Nem precisam estar online juntos**.

O broker (RabbitMQ) desacopla em três eixos:

► **No tempo**

a fila é durável: o produtor publica agora, o consumidor pega depois — pode nem estar ligado

► **No espaço**

ninguém sabe endereço nem porta de ninguém; todos sabem só do broker

► **Em linguagem e protocolo**

nosso back-end fala **AMQP** (C#) e o navegador fala **STOMP/WebSocket** (JavaScript) — **nas mesmas filas**, porque o broker traduz

Por que Fila e não Publicar/Assinar?

Escolhemos a **Opção A — Fila de Mensagens** (ponto-a-ponto / *work queue*)

FILA · o que o problema pede

Cada partícula tem que ser desenhada **por UM consumidor só**.

A fila **reparte** as 1.328 entre quem estiver ativo.

Se dois desenharem o mesmo pixel, o boss sairia **duplicado** em vez de **repartido**.

PUB/SUB · o que aconteceria

Cada assinante receberia **uma cópia de TODAS** as partículas — o boss inteiro **em cada um**, em vez de dividido.

Trocaríamos a fila por uma **exchange fanout**, e cada assinante teria a **sua própria fila**.

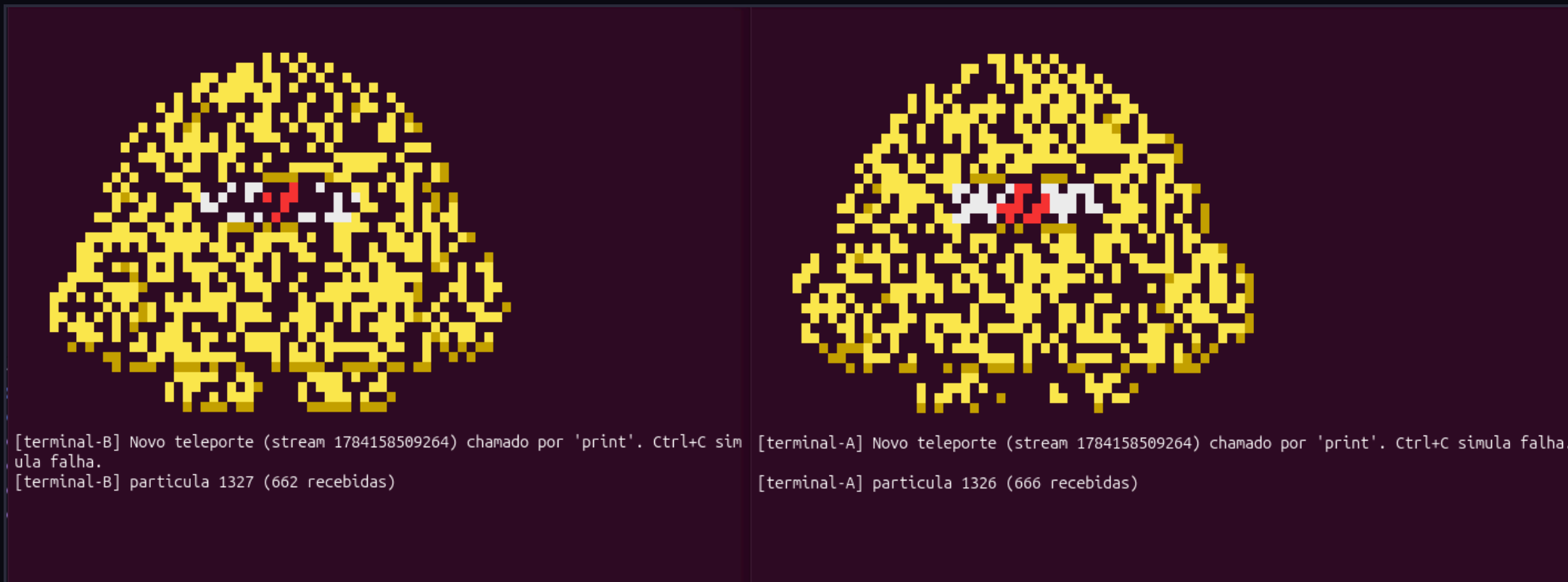
A frase que resume: pub/sub serve para **notificar todo mundo**. Nós precisávamos **repartir tarefas** — e isso é fila.

Como o MOM funciona aqui



Outras filas: devil-hits (cada tiro) e devil-return (devolver o boss). O navegador **publica nelas** — ele é **produtor E consumidor ao mesmo tempo**.

Cada mensagem para um só consumidor



$$666 + 662 = 1.328$$

o total exato do sprite

Os dois terminais estão **no mesmo teleporte** e cada um desenhou **só a metade que pegou** — por isso o boss aparece chuviscado nos dois. **Nenhuma partícula foi processada duas vezes, nenhuma se perdeu.**

O que a Management UI mostra

 RabbitMQ 4.0.5 Erlang 27.3.4.6

Refreshed 2026-07-15 20:19:56 Refresh every 5 seconds

Virtual host All

Cluster **rabbit@haji**

User **guest** Log out

△ All stable feature flags must be enabled after completing an upgrade. [\[Learn more\]](#)

Overview Connections Channels Exchanges Queues and Streams Admin

Queues

▼ All queues (4)

Pagination

Page 1 of 1 - Filter: ☐ Regexp ? Displaying 4 items , page size up to: 100

Overview					Messages			Message rates		
Virtual host	Name	Type	Features	State	Ready	Unacked	Total	incoming	deliver / get	ack
/	devil-hits	classic	D Args	idle	0	0	0			
/	devil-particles	classic	D Args	running	544	2	546	0.00/s	22/s	22/s
/	devil-return	classic	D Args	idle	0	0	0			
/	devil-summons	classic	D Args	idle	0	0	0	0.00/s	0.00/s	0.00/s

As 4 filas, todas com o selo **D** = **duráveis**. devil-particles em **running**, entregando a **22/s**.

Unacked = 2 — a prova do prefetch=1: cada um dos **2 consumidores** segura **uma única** mensagem por vez.

Quatro coisas, ao vivo

- ▶ **1 · A divisão** — dois terminais + o navegador disputando a fila
o boss se reparte entre os três; cada partícula aparece **num lugar só**
- ▶ **2 · A falha e a reentrega** — Ctrl+C num consumidor no meio do teleporte
as mensagens que ele **não confirmou** voltam para a fila e são **reentregues** ao outro: aparece << **REENTREGUE** apos **falha!**. **Nada se perde.**
- ▶ **3 · Entra e sai consumidor** — feche um terminal e invoque de novo
o que sobrou recebe **as 1.328** e monta o boss **inteiro** — a fila **redistribui a carga sozinha**, sem ninguém reconfigurar o produtor
- ▶ **4 · O front é produtor também** — ESPAÇO atira no boss
o tiro vai para **devil-hits** e o **HP autoritativo** cai no console do orquestrador: o navegador **publica** e **consome** pelo mesmo broker

Como executar e conectar — MOM

```
# 1) o broker — UMA VEZ SÓ na vida da máquina
$ ./devil.sh broker          # apt + plugins (Linux)
> .\devil.ps1 broker        # winget (Windows, como ADMIN)

# 2) a demo — abre as 4 abas + o navegador
$ ./devil.sh mom

# 3) invocar sem o navegador (plano B)
$ ./devil.sh invocar

$ ./devil.sh status          # o que está de pé
$ ./devil.sh stop            # derruba as demos
```

Sem Docker: o RabbitMQ roda **nativo**, como serviço do sistema — sobe junto com a máquina.

COMO CADA UM CONECTA

Porta	Quem usa
5672	back-end C# → broker (AMQP)
15674	front JS → broker (STOMP/WS)
15672	Management UI — guest/guest
8080	a página do front (http.server)

Ninguém configura endereço de ninguém. Cada componente aponta para o **broker** e diz o **nome da fila**. Só.

O mesmo boss, os dois paradigmas

	gRPC	MOM (fila)
Acoplamento	cliente conhece o endereço do servidor	só o nome da fila
Tempo	síncrono — os dois online juntos	assíncrono — a fila durável guarda
Entrega	1 cliente ↔ 1 servidor	N disputam ; cada msg para um
Contrato	.proto tipado , stub gerado	JSON por convenção
Se o outro cai	o stream quebra	a msg volta pra fila e é reentregue
Serve para	chamada remota tipada e rápida	repartir trabalho e desacoplar

Obrigado! — perguntas?